

Raport privind Utilizarea Instrumentelor de Inteligență Artificială în Dezvoltarea Software

1. Introducere & Proporția de Colaborare

Pentru dezvoltarea proiectului Atelier AI, am împărțit sarcinile în funcție de eficiență, stabilind o colaborare clară între echipa noastră și instrumentele AI. Ne-am asumat rolul de Arhitect și Product Owner (aproximativ 30% din efort), ocupându-ne de viziune, logica de business, structura sistemului și rezolvarea erorilor critice. Instrumentele AI au preluat rolul de executant (restul de 70%), fiind folosite pentru generarea rapidă de cod de bază (boilerplate), stilizare CSS și scrierea testelor automate.

Această abordare ne-a permis să ridicăm de la zero o aplicație completă (frontend în React cu Zustand, backend în Django REST Framework și integrare LLM local prin Ollama) într-un timp scurt, fără să ne blocăm în sarcini repetitive de configurare.

2. Ce a fost conceput exclusiv de ECHIPA UMANĂ (30% Arhitectură și Viziune)

AI-ul tinde adesea să ofere soluții standardizate sau prea complexe. Pentru ca aplicația să aibă o logică solidă și un flux fluid, deciziile critice au fost luate exclusiv de noi:

A. Decizia arhitecturală: AI rulat local (Ollama)

În loc să apelăm la API-uri externe plătite, am decis să integrăm Ollama pentru a rula modele direct pe calculatorul utilizatorului. A fost o decizie strategică esențială: am eliminat costurile cu tokenii și am asigurat confidențialitatea datelor, totul funcționând offline.

B. Simplificarea fluxului de utilizare

Modelele AI ne propuneau inițial un asistent clasic împărțit în foarte mulți pași separați. Noi am restructurat procesul și l-am redus la un flux rapid de 4 pași: dimensiunile camerei, chatul interactiv pentru stil și buget, generarea layout-ului 2D și exportul final. De asemenea, am implementat manual logica de reținere a pasului curent în browser, pentru ca utilizatorul să nu-și piardă progresul la un refresh accidental.

C. Direcția vizuală și interfața premium

Am refuzat temele vizuale standard pe care AI-ul le generează de obicei. Am gândit de la zero o estetică minimalistă, potrivită pentru un studio de arhitectură: fundal închis, accente aurii și o grilă tehnică curată. Comportamentul fluid al butoanelor la trecerea mouse-ului și efectele de transparență au fost ajustate direct de noi pentru a obține aspectul dorit.

D. Logica pentru strategiile de așezare în spațiu

Când am testat generarea automată a pozițiilor, AI-ul oferea soluții haotice sau obiecte suprapuse. Noi am conceput structura logică din backend bazată pe 8 strategii geometrice clare (aranjare pe colțuri, distribuție pe perimetru, grupare centrală etc.), obligând sistemul să

returneze configurații complet diferite la fiecare încercare.

E. Mecanismul de fallback pentru spații mici

Ne-am pus problema unui caz limită: ce se întâmplă dacă utilizatorul introduce o cameră foarte mică, iar piesele standard din baza de date nu încap? Am creat o logică în backend care forțează sistemul să relaxeze filtrele și să aleagă cel mai mic mobilier disponibil, instruind în același timp chatul să explice politicos această constrângere fizică.

F. Corectarea exportului PDF

La generarea raportului, diacriticele românești apăreau corupte din cauza limitărilor de codare ale bibliotecii jsPDF. AI-ul încerca să rezolve problema prin scripturi complicate care încărcau fonturi externe masive. Soluția noastră a fost mult mai simplă: am creat o funcție proprie care transformă automat caracterele speciale în variantele lor simple (de exemplu, ș devine s), asigurând un text lizibil.

3. Ce a fost delegat și implementat de AI (70% Execuție și Codare)

Odată stabilite regulile și arhitectura, am folosit inteligența artificială pentru a accelera scrierea codului propriu-zis și a componentelor repetitive:

- **Structura de bază (Boilerplate):** AI-ul a generat rapid scheletul pentru aplicațiile Django, configurarea inițială pentru Django REST Framework, modelele de bază și structura de pornire a store-ului Zustand din React.
- **Scrierea CSS-ului:** După ce i-am explicat conceptul vizual (culori, efecte de sticlă), AI-ul a scris rapid codul CSS brut, inclusiv regulile de așezare în pagină și adaptabilitatea interfeței pentru ecranele de mobil sau tabletă.
- **Generarea testelor automate:** I-am cerut AI-ului să scrie suita de teste în Vitest pe baza unor scenarii logice stabilite de noi (de exemplu, validarea faptului că prețul total al mobilierului recomandat nu depășește bugetul setat).

4. Metodologia de lucru și controlul calității

Pentru a menține controlul asupra proiectului și a nu lăsa AI-ul să genereze cod inutil, am aplicat câteva reguli de lucru:

1. **Abordare modulară:** Nu am cerut niciodată cod pentru întreaga aplicație deodată. Am transmis AI-ului doar funcțiile izolate sau componentele specifice la care lucram în acel moment.
2. **Corecție directă:** Atunci când codul propus de AI pentru algoritmul de poziționare era inefficient, am intervenit, i-am explicat unde greșește logica și i-am impus să folosească structura noastră de strategii geometrice.
3. **Formatare strictă pentru date:** Pentru ca backend-ul să poată procesa stabil răspunsurile de la modelul local Ollama, am configurat instrucțiuni stricte care au forțat AI-ul să răspundă exclusiv în format JSON curat, eliminând textul suplimentar inutil.

5. Concluzii

Dezvoltarea proiectului Atelier AI a arătat că utilizarea inteligenței artificiale este extrem de

eficientă dacă echipa umană păstrează controlul arhitectural. Delegând sarcinile consumatoare de timp către AI (cod de bază, structuri CSS, configurări de teste), ne-am putut concentra resursele pe logica algoritmilor de așezare, designul personalizat și stabilitatea aplicației. Rezultatul este un produs complet funcțional și bine structurat.